UADE

UADE

Programación III – Diseño y Análisis de Algoritmos

Temas a desarrollar

1	Repaso Poda por cota inferior y/o superior
2	Problema del laberinto
3	Asignación de Tareas

Repaso clase anterior

¿Qué es una cota?

- En backtracking, las **cotas** (bounds su termino en ingles) se uilizan para decidir si conviene seguir explorando una rama del árbol de búsqueda o podarla (descartarla). Son estimaciones del valor que puede alcanzar una solución parcial que se esta construyendo.

- En general se van a utilizar dos tipos de cota, inferior y superior.

Backtracking

Cota Inferior

Es un límite mínimo del valor que puede alcanzar una solución que se extiende desde una solución parcial:

- Representa el mejor caso posible que podrías lograr continuando por esa rama.
- En problemas de minimización, indica el menor costo posible que podría llegar a tener esa rama.
- Si esa cota inferior ya es peor que la mejor solución encontrada hasta el momento, se poda la rama porque no vale la pena seguir.

Backtracking

Cota Superior

Es un límite máximo del valor que puede alcanzar una solución en esa rama:

- ✓ Representa una estimación optimista del mejor valor posible a alcanzar (aunque no sea el real).
- ✓ En problemas de maximización, indica el máximo beneficio posible que podría lograrse desde esa rama.
- ✓ Si esa cota superior es menor que la mejor solución ya encontrada, también se puede podar.

Ejercicio Práctica

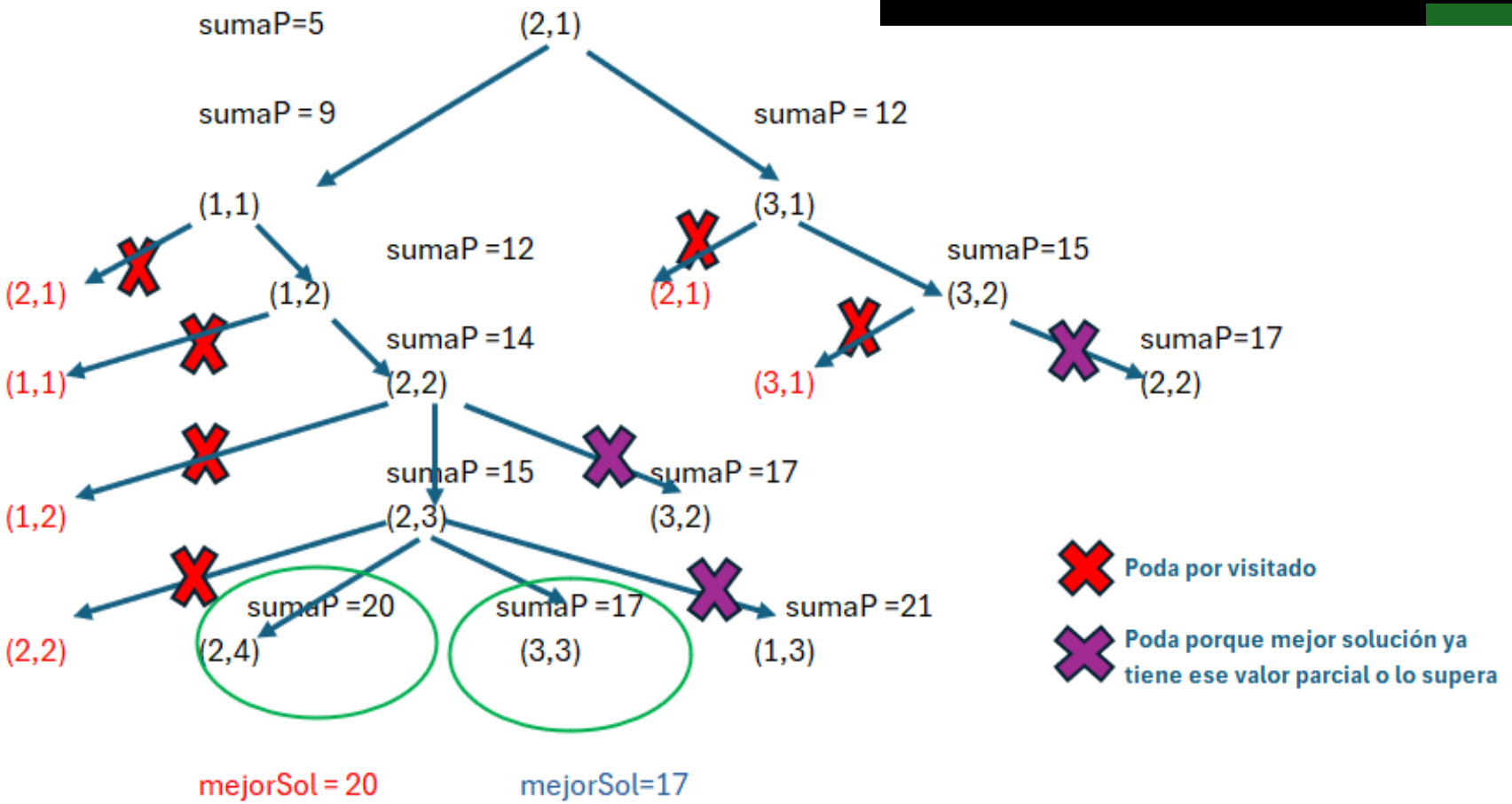
Ejercicio

- Plantear la estrategia por backtracking
- Desarrollar pseudocódigo
- Calcular complejidad temporal teórica

Ejercicio 2. Dado un laberinto consistente en una matriz cuadrada que tiene en cada posición ur valor natural y cuatro valores booleanos, indicando estos últimos si desde esa casilla se puede ir al norte, este, sur y oeste, encontrar un camino de longitud mínima entre dos casillas dadas, siendo la longitud de un camino la suma de los valores naturales de las casillas por las que pasa.

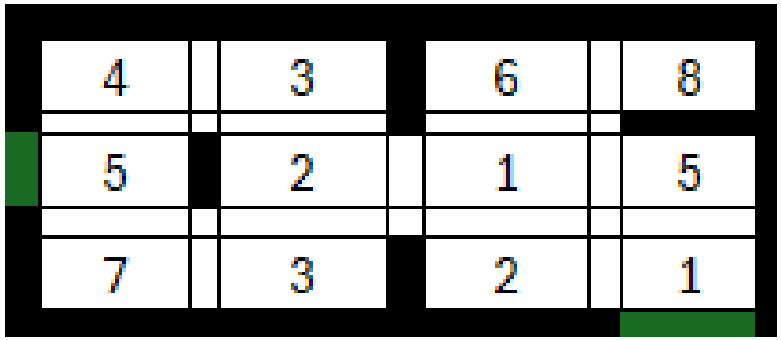
Problema del laberinto- Estrategia

4	3	6	8
5	2	1	5
7	3	2	1



UADE

Problema del laberinto- Estrategia Cont.



- La matriz de entrada se puede representar como una matriz en donde cada posición tiene: valor natural, booleano1, booleano2, booleano3, booleano4. Cada booleano representa si se puede mover a la izquierda, arriba, abajo o derecha, respectivamente.
- Deben tener un control de visitados, puede ser una matriz booleana.
- La matriz de visitados se puede copiar en una matriz resultado, cuando se queda como mejor solución.

Problema del laberinto- Tarea

- ✓ A partir de la estrategia anterior escribir el pseudocódigo correspondiente.

- ✓ Calcular la complejidad temporal.

Problema del laberinto- Pseudocódigo

- ✓ Consideraciones:

Celda:
- valor: ENTERO
- arriba: BOOLEANO
- abajo: BOOLEANO
- izq: BOOLEANO
- der: BOOLEANO

Posicion:
- i: ENTERO
- j: ENTERO

Problema del laberinto- Pseudocódigo

```
Algoritmo Laberinto
Entradas: Matriz<celda> laberinto, entero n, Posicion posActual, Posicion destino,
Matriz<booleano> visitado, Matriz<booleano> resultado, entero sumaP, entero mejorSuma
Salida: --
  Entero i ← posActual.i
  Entero j← posActual.j
  visitado[i][j] ← verdadero
  sumaP ← sumaP + laberinto[i][j].valor
  Si i = destino.i Y j = destino.j
    Si sumaP < mejorSuma
        mejorSuma ← sumaP
        copiar(resultado, visitado)
    fin si
 fin si
 sino
  Si sumaP < mejorSuma
          Posicion nuevaPos
           Si laberinto[i][j].arriba = verdadero Y visitado[i-1][j] = falso
              nuevaPos.i ← i - 1
              nuevaPos.j ← j
              backtrack(laberinto, n, nuevaPos, destino, visitado, resultado, sumaP, mejorSuma)
           fin si
           Si laberinto[i][j].abajo =verdadero Y visitado[i+1][j] = falso
              nuevaPos.i ← i + 1
              nuevaPos.j ← j
              backtrack(laberinto, n, nuevaPos, destino, visitado, resultado, sumaP, mejorSuma)
           fin si
           Si laberinto[i][j].izq =verdadero Y visitado[i][j-1] = falso
              nuevaPos.i ← i
              nuevaPos.j ← j - 1
              backtrack(laberinto, n, nuevaPos, destino, visitado, resultado, sumaP, mejorSuma)
           fin si
          Si laberinto[i][j].der =verdadero Y visitado[i][j+1] = falso
              nuevaPos.i ← i
              nuevaPos.j ← j + 1
              backtrack(laberinto, n, nuevaPos, destino, visitado, resultado, sumaP, mejorSuma)
           fin si
fin sino
visitado[i][j] ← falso
sumaP ← sumaP + laberinto[i][j].valor
fin laberinto
```

Laberinto- Complejidad Temporal

```
Algoritmo Laberinto
Entradas: Matriz<celda> laberinto, entero n, Posicion posActual, Posicion destino,
Matriz<booleano> visitado, Matriz<booleano> resultado, entero sumaP, entero mejorSuma
Salida: --
  Entero i ← posActual.i
  Entero j← posActual.j
  visitado[i][j] ← verdadero
  sumaP ← sumaP + laberinto[i][j].valor
  Si i = destino.i Y j = destino.j
    Si sumaP < mejorSuma
        mejorSuma ← sumaP
        copiar(resultado, visitado)
    fin si
  fin si
  sino
  Si sumaP < mejorSuma
          Posicion nuevaPos
           Si laberinto[i][j].arriba = verdadero Y visitado[i-1][j] = falso
              nuevaPos.i ← i - 1
              nuevaPos.j ← j
              backtrack(laberinto, n, nuevaPos, destino, visitado, resultado, sumaP, mejorSuma)
           fin si
           Si laberinto[i][j].abajo =verdadero Y visitado[i+1][j] = falso
              nuevaPos.i ← i + 1
              nuevaPos.j ← j
              backtrack(laberinto, n, nuevaPos, destino, visitado, resultado, sumaP, mejorSuma)
           fin si
           Si laberinto[i][j].izq =verdadero Y visitado[i][j-1] = falso
              nuevaPos.i ← i
              nuevaPos.j ← j - 1
              backtrack(laberinto, n, nuevaPos, destino, visitado, resultado, sumaP, mejorSuma)
           fin si
          Si laberinto[i][j].der =verdadero Y visitado[i][j+1] = falso
              nuevaPos.i ← i
              nuevaPos.j ← j + 1
              backtrack(laberinto, n, nuevaPos, destino, visitado, resultado, sumaP, mejorSuma)
           fin si
fin sino
visitado[i][j] ← falso
sumaP ← sumaP + laberinto[i][j].valor
fin laberinto
```

a = 3

b = 1

k = 0

Caso Resta

O(3^n) siendo n cantidad de celdas

Asignación de tareas

UADE

Asignación de tareas

Las reglas de juego son las siguientes:

- Hay ***m*** personas para ***m*** tareas. Cualquiera de ellas podría realizar cualquiera de las tareas.

- Cada asignación de tareas implica un costo, dependiendo de la tarea y de la persona a quien se la asignó.

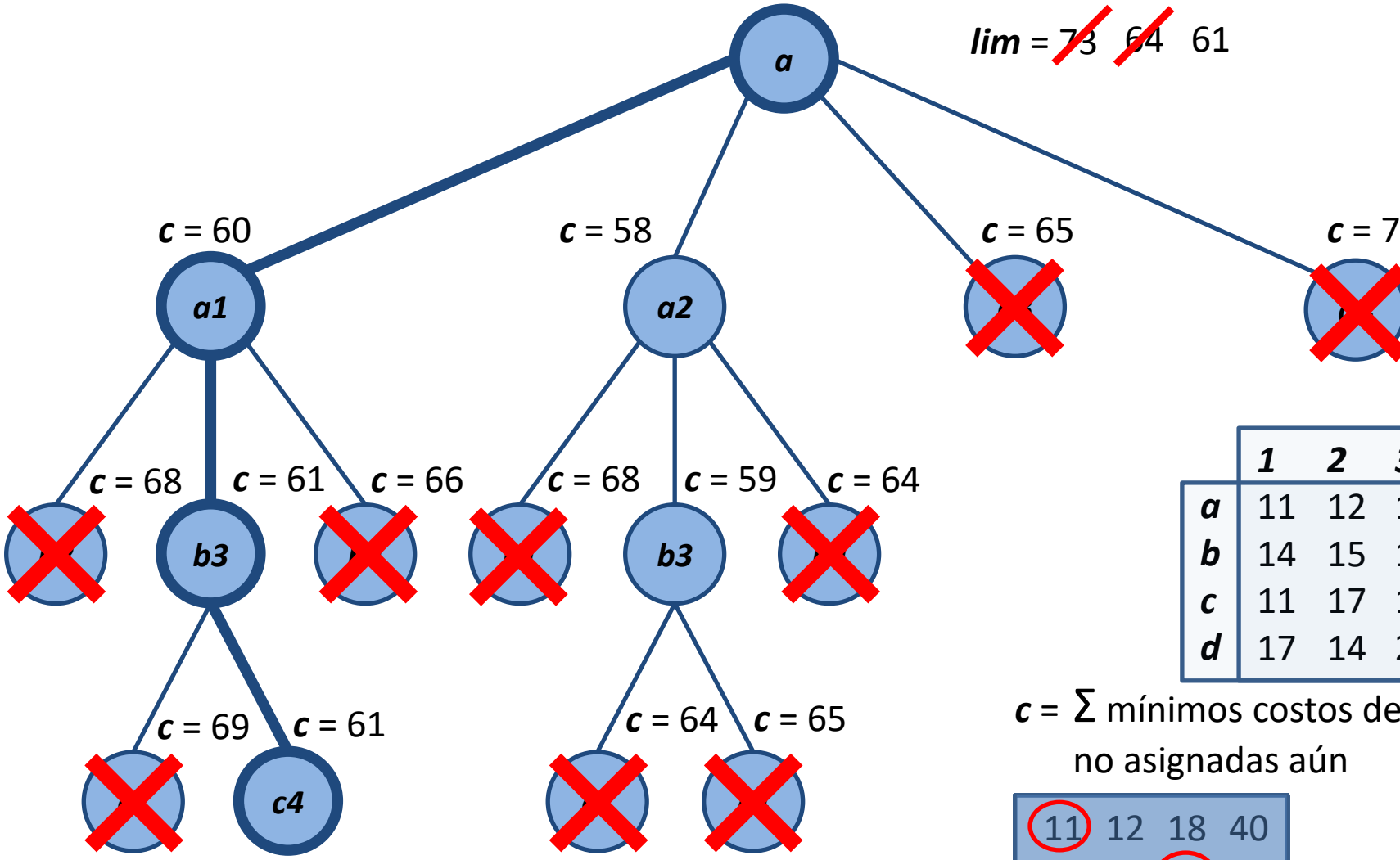
- Es necesario que se realicen todas las tareas, que cada tarea esté asignada a una persona y que cada persona realice sólo una tarea.

- Se quiere minimizar el costo total de las asignaciones de tareas.

Asignación de tareas. Estrategia.

- Para resolver este problema exploraremos un árbol en el que cada nodo corresponde a una asignación de una tarea a una persona.
- En cada nodo se calcula un costo. El costo se calcula como la suma de los costos actuales más los mínimos valores de las tareas no asignadas
- Un límite superior se puede obtener con una asignación arbitraria. Una solución óptima no puede ser mayor. Si una solución es menor que el límite superior, lo reemplaza. Este límite será la variable ***lim***.
- Este proceso continúa: si se asigna la tarea ***1*** a la persona ***a***, entonces se suman los mínimos valores de la columna para cada una de las restantes tareas, excluyendo la fila de ***a***.

Ejemplo: asignación de tareas.



	1	2	3	4
a	11	12	18	40
b	14	15	13	22
c	11	17	19	23
d	17	14	20	28

c = Σ mínimos costos de tareas no asignadas aún

11	12	18	40
14	15	13	22
11	17	19	23
17	14	20	28

UADE

Asignación de tareas: Pseudocódigo

```
Algoritmo BackTareas
Entradas: costo Matriz<Entero>, m: Entero, tarea: Entero, costo_actual: Entero, usado: Vector<Booleano>,
asignacion_actual: Vector<Entero>, mejor_asignacion: Vector<Entero>, var lim: Entero

  si tarea = m entonces
    si costo_actual < lim
      lim ← costo_actual
      mejor_asignacion ← copiar(asignacion_actual)
    fin si
fin si
sino
  estimado: Entero ← costo_actual + COSTO_MIN_RESTANTE(costo, m, tarea, usado)
    si estimado < lim
            para persona: entero desde 0 hasta m-1
              si usado[persona] = falso
                usado[persona] ← verdadero
                asignacion_actual[tarea] ← persona
                BackTareas (costo,m, tarea + 1, costo_actual + costo[persona][tarea], usado,
          asignacion_actual,  mejor_asignacion, lim)
                usado[persona] ← falso
              fin si
            fin para
     fin si
fin sino
fin Algoritmo
```

Asignación de tareas: Pseudocódigo

```
Algoritmo COSTO_MIN_RESTANTE
Entradas: costo: Matriz<Entero>, m: Entero, tarea_actual: Entero, usado: Vector<Booleano>
Salida: valor: Entero

  suma: Entero ← 0
  para t:entrero desde tarea_actual hasta m-1
    minimo: Entero ← +infinito
    para persona:entero desde 0 hasta m-1
      si usado[persona] = falso
        minimo ← min(minimo, costo[persona][t])
      fin si
    fin para
    suma ← suma + minimo
  fin para
  devolver suma
```

Asignación de tareas: Pseudocódigo

```
Algoritmo CALCULAR_SOLUCION_INICIAL
Entradas: costo: Matriz<Entero>,   m: Entero
Salidas: valor: Entero
   total: Entero ← 0
   usado: Vector<booleano>
   para i:entero desde 0 hasta m-1
      usado[i] ← falso
   fin para
   para tarea: Entero desde 0 hasta m-1
      persona: Entero ← -1
      salir : booleano ←falso
     para i:entero desde 0 hasta m-1 y salir =falso
        si usado[i] = falso
           persona ← i
           salir=verdadero
        fin si
      fin para
      usado[persona] ← verdadero
      total ← total + costo[persona][tarea]
   fin para
   devolver total
```

Asignación de tareas: Complejidad Temporal

```
Algoritmo BackTareas
Entradas: costo Matriz<Entero>, m: Entero, tarea: Entero, costo_actual: Entero, usado: Vector<Booleano>,
asignacion_actual: Vector<Entero>, mejor_asignacion: Vector<Entero>, var lim: Entero

  si tarea = m entonces
    si costo_actual < lim
      lim ← costo_actual
      mejor_asignacion ← copiar(asignacion_actual)
    fin si
fin si
sino
  estimado: Entero ← costo_actual + COSTO_MIN_RESTANTE(costo, m, tarea, usado)
    si estimado < lim
            para persona: entero desde 0 hasta m-1
              si usado[persona] = falso
                usado[persona] ← verdadero
                asignacion_actual[tarea] ← persona
                BackTareas (costo,m, tarea + 1, costo_actual + costo[persona][tarea], usado,
          asignacion_actual,  mejor_asignacion, lim)
                usado[persona] ← falso
              fin si
            fin para
     fin si
fin sino
fin Algoritmo
```

a= personas

b = 1

k = no influye, es personas x tares

n = tareas

Caso resta → **O(p^n) = O(n^n)**

Aunque se puede deducir → **O(n!)**

UADE

¡Muchas gracias!

UADE